

FCFS and SJF Scheduling

Operating System Lab (5 Marks)

Name: Aniket Singh | Roll No: 2501012308 | Section: E

Environment: WSL (Ubuntu) | Language: Python 3

Task-wise Summary

#	Task	Implementation
1	Accept PID, Arrival, Burst time	read_processes()
2	Validate input values	Integer check, AT >= 0, BT >= 1, unique non-empty PID; re-prompts on error
3	Preserve original data	copy.deepcopy(); final check prints 'Original data preserved: YES'
4	FCFS by arrival order	Sort by (arrival time, input order)
5	Non-preemptive SJF	Pick shortest burst among arrived processes
6	Tie-breaking rule	Earlier arrival first, then input order (smaller PID)
7	Display CPU idle intervals	IDLE[start-end] in sequence and idle interval line
8	Separate sequences and intervals	Separate FCFS and SJF sections with Gantt chart
9	Same dataset for both	Both algorithms receive copies of the same dataset

Dataset and Results

PID	AT	BT
P1	0	7
P2	2	4
P3	4	1
P4	5	4
P5	20	3

Algorithm	Sequence	Avg TAT	Avg WT
FCFS	P1, P2, P3, P4, IDLE(16-20), P5	7.60	3.80
SJF	P1, P3, P2, P4, IDLE(16-20), P5	7.00	3.20

Tie-breaking rule: earlier arrival time first; if still tied, the process entered first (smaller PID order) runs first. **Conclusion:** SJF gives lower average waiting time (3.20) than FCFS (3.80) for this dataset, and both show the CPU idle interval 16-20.

Execution Evidence

```
thean@MSI:/mnt/c/Users/thean$ cd ~/oslab
cp /mnt/c/Users/thean/Downloads/fcfs_sjf.py .
python3 fcfs_sjf.py
cp: cannot stat '/mnt/c/Users/thean/Downloads/fcfs_sjf.py': No such file or directory
python3: can't open file '/home/thean/oslab/fcfs_sjf.py': [Errno 2] No such file or directory
thean@MSI:~/oslab$ ls /mnt/c/Users/thean/Downloads | grep -i fcfs
thean@MSI:~/oslab$ cd ~/oslab
cp /mnt/c/Users/thean/Downloads/fcfs_sjf.py .
python3 fcfs_sjf.py
Enter number of processes: 0
  Invalid count: must be >= 1. Try again.
Enter number of processes: abc
  Invalid count: 'abc' is not an integer. Try again.
Enter number of processes: 5
Process 1:
  Process ID (e.g. P1): p1
  Arrival Time (>= 0): 0
  Burst Time (>= 1): 7
Process 2:
  Process ID (e.g. P1): p1
  Invalid Process ID: 'p1' already used. Try again.
  Process ID (e.g. P1): p2
  Arrival Time (>= 0): -2
  Invalid Arrival Time: must be >= 0. Try again.
  Arrival Time (>= 0): 2
  Burst Time (>= 1): 0
  Invalid Burst Time: must be >= 1. Try again.
  Burst Time (>= 1): 4
Process 3:
  Process ID (e.g. P1): p3
  Arrival Time (>= 0): 4
  Burst Time (>= 1): 1
Process 4:
  Process ID (e.g. P1): p4
  Arrival Time (>= 0): 5
  Burst Time (>= 1): 4
Process 5:
  Process ID (e.g. P1): p5
  Arrival Time (>= 0): 20
  Burst Time (>= 1): 3

Original process data (same dataset used for both algorithms):
PID    AT    BT
p1      0     7
p2      2     4
p3      4     1
p4      5     4
p5     20     3
```

Screenshot 1: Input and validation - invalid count (0, abc), duplicate PID, negative arrival time and zero burst time are rejected; valid dataset accepted and original data displayed (Tasks 1, 2, 3, 9)

```
=====
FCFS Scheduling
=====
Execution sequence:
  p1[0-7] -> p2[7-11] -> p3[11-12] -> p4[12-16] -> IDLE[16-20] -> p5[20-23]

Gantt chart:
|   p1 |   p2 |   p3 |   p4 | IDLE |   p5 |
|   0   |   7   |  11  |  12  |  16  |  20  |  23

CPU idle intervals: 16-20

PID    AT  BT  Start  End  TAT  WT
p1      0   7     0    7    7   0
p2      2   4     7   11    9   5
p3      4   1    11   12    8   7
p4      5   4    12   16   11   7
p5     20   3    20   23    3   0

Average TAT = 7.60
Average WT  = 3.80
```

Screenshot 2: FCFS - execution sequence, Gantt chart, CPU idle interval 16-20, and table with Average WT = 3.80 (Tasks 4, 7, 8)

```
=====
SJF (Non-preemptive) Scheduling
=====
Execution sequence:
  p1[0-7] -> p3[7-8] -> p2[8-12] -> p4[12-16] -> IDLE[16-20] -> p5[20-23]

Gantt chart:
|   p1 |   p3 |   p2 |   p4 | IDLE |   p5 |
|   0   |   7   |   8   |  12  |  16  |  20  |  23

CPU idle intervals: 16-20

PID    AT  BT  Start  End  TAT  WT
p1      0   7     0    7    7   0
p3      4   1     7    8    4   3
p2      2   4     8   12   10   6
p4      5   4    12   16   11   7
p5     20   3    20   23    3   0

Average TAT = 7.00
Average WT  = 3.20

Original data preserved: YES
thean@MSI:~/oslab$ |
```

Screenshot 3: Non-preemptive SJF - execution sequence, Gantt chart, CPU idle interval 16-20, Average WT = 3.20, original data preserved (Tasks 3, 5, 7, 8)

Source Code: fcfs_sjf.py

```
"""FCFS and Non-preemptive SJF CPU Scheduling.

Tie-breaking rule (both algorithms):
1. Earlier arrival time first
2. If still tied, smaller Process ID (input order) first
"""
import copy

# ----- Task 1 & 2: Input with validation -----
def read_int(prompt, minimum, label):
    while True:
        raw = input(prompt).strip()
        try:
            value = int(raw)
        except ValueError:
            print(f" Invalid {label}: '{raw}' is not an integer. Try again.")
            continue
        if value < minimum:
            print(f" Invalid {label}: must be >= {minimum}. Try again.")
            continue
        return value

def read_processes():
    n = read_int("Enter number of processes: ", 1, "count")
    processes, seen = [], set()
    for i in range(1, n + 1):
        print(f"Process {i}:")
        while True:
            pid = input(" Process ID (e.g. P1): ").strip()
            if not pid:
                print(" Invalid Process ID: cannot be empty. Try again.")
            elif pid in seen:
                print(f" Invalid Process ID: '{pid}' already used. Try again.")
            else:
                seen.add(pid)
                break
        at = read_int(" Arrival Time (>= 0): ", 0, "Arrival Time")
        bt = read_int(" Burst Time (>= 1): ", 1, "Burst Time")
        processes.append({"pid": pid, "at": at, "bt": bt, "order": i})
    return processes

# ----- Scheduling core -----
def fcfs(procs):
    procs = sorted(procs, key=lambda p: (p["at"], p["order"])) # Task 4
    time, timeline, result = 0, [], []
    for p in procs:
        if time < p["at"]:
            timeline.append(("IDLE", time, p["at"])) # Task 7
            time = p["at"]
        start, end = time, time + p["bt"]
        timeline.append((p["pid"], start, end))
        result.append(make_row(p, start, end))
        time = end
    return timeline, result

def sjf(procs):
    remaining = list(procs)
    time, timeline, result = 0, [], []
    while remaining:
        ready = [p for p in remaining if p["at"] <= time]
        if not ready: # Task 7
            next_at = min(p["at"] for p in remaining)
            timeline.append(("IDLE", time, next_at))
            time = next_at
            continue
        # Task 5 & 6: shortest burst, tie -> earlier arrival -> smaller order
        p = min(ready, key=lambda x: (x["bt"], x["at"], x["order"]))
        start, end = time, time + p["bt"]
        timeline.append((p["pid"], start, end))
        result.append(make_row(p, start, end))
        remaining.remove(p)
        time = end
    return timeline, result

def make_row(p, start, end):
    tat = end - p["at"]
```

```

        return {"pid": p["pid"], "at": p["at"], "bt": p["bt"],
                "start": start, "end": end, "tat": tat, "wt": tat - p["bt"]}

# ----- Task 8: Display -----
def show(title, timeline, result):
    print("\n" + "=" * 62)
    print(f"{title}")
    print("=" * 62)
    print("Execution sequence:")
    print(" " + " " -> ".join(f"{n} [{s}-{e}]" for n, s, e in timeline))
    print("\nGantt chart:")
    bar = "|" + "|".join(f" {n} ".center(7) for n, _, _ in timeline) + "|"
    marks = f"{timeline[0][1]:<8}" + "".join(f"e:<8}" for _, _, e in timeline)
    print(" " + bar)
    print(" " + marks)
    idle = [(s, e) for n, s, e in timeline if n == "IDLE"]
    if idle:
        print("\nCPU idle intervals: " + ", ".join(f"{s}-{e}" for s, e in idle))
    else:
        print("\nCPU idle intervals: None")
    print("\n{<6}{>4}{>4}{>7}{>5}{>5}{>5}.format(
        "PID", "AT", "BT", "Start", "End", "TAT", "WT"))
    for r in sorted(result, key=lambda r: r["start"]):
        print("{<6}{>4}{>4}{>7}{>5}{>5}{>5}.format(
            r["pid"], r["at"], r["bt"], r["start"], r["end"], r["tat"], r["wt"]))
    n = len(result)
    print(f"\nAverage TAT = {sum(r['tat'] for r in result) / n:.2f}")
    print(f"Average WT = {sum(r['wt'] for r in result) / n:.2f}")

def main():
    original = read_processes()
    backup = copy.deepcopy(original) # Task 3: preserve original data

    print("\nOriginal process data (same dataset used for both algorithms):")
    print("{<6}{>4}{>4}.format("PID", "AT", "BT"))
    for p in original:
        print("{<6}{>4}{>4}.format(p["pid"], p["at"], p["bt"]))

    # Task 9: same dataset passed (as independent copies) to both algorithms
    t1, r1 = fcfs(copy.deepcopy(original))
    t2, r2 = sjf(copy.deepcopy(original))

    show("FCFS Scheduling", t1, r1)
    show("SJF (Non-preemptive) Scheduling", t2, r2)

    print("\nOriginal data preserved:", "YES" if original == backup else "NO")

if __name__ == "__main__":
    main()

```